The Instrument

A Software-Engineering Primer on How the Research Codebase Is Built, Run, and Kept Honest

A companion to *The Mathematics of Constraint*.

This repository is not an application. It is a scientific instrument and a publishing pipeline — engineered to keep machine-made science honest.

Reading a Codebase as an Argument

Most software exists to *do* something — serve a page, process a payment, drive a car. This codebase exists to *establish* something: trustworthy, reproducible scientific records, generated by AI agents under one human director's review. Almost every engineering decision in it — the way raw data is split from published summaries, the script that regenerates provenance, the guard that blocks unsafe commits, the pinned GPU images — is downstream of a single requirement: **how do you keep science honest when a machine wrote the code and a machine ran the experiment?**

Read this way, the repository becomes legible. Its unusual choices stop looking arbitrary and start looking like answers to that question. This book is a guided tour of the instrument — its architecture, how it runs experiments locally and on rented GPUs, how results become papers, and the quality and provenance machinery that lets a suspicious outsider verify a result without trusting the AI that produced it. It closes, as an honest engineering review must, with the smells, the tech debt, and a concrete plan for what to refactor.

Who this is for

You need to know what a function, a file, and a commit are — roughly, that you have written or read some code. You do *not* need to know research infrastructure, GPUs, or the specific tools (Modal, Doppler, Haskell, reportlab) involved; each is explained where it appears. Boxes work as in the other primers: Definition , Intuition pump , In the code (pointing at real files), and Smell (where an engineer should wince, constructively).

Contents

PART I

The Shape of the Codebase

Before the running and the tooling comes the layout — what lives where, and why. This part maps the repository's structure, its central "experiment package" convention, and the safety split that keeps it publishable. The organizing insight, which unlocks everything: this is an instrument for making honest records, not an app for doing a task.

CHAPTER 1

Not an App — a Scientific Instrument

1.1 The framing that explains everything

The single most useful thing to know before opening a single file: **this is not an application.** An application has users, features, a runtime it serves. This repository has none of those as its purpose. Its "product" is a body of *scientific records* — experiment code, result summaries, papers, and the provenance that proves how they were made — that must remain observable, attributable, and reproducible even though an AI agent generated them. Every design choice that would look strange in an app makes sense once you hold this frame.

Intuition pump — a lab bench, not a product

Walk into a chemistry lab and you do not find "an app." You find benches, labeled reagent shelves, a logbook, a fume hood, and a strict protocol for what may leave the room. The organization serves *reproducible, safe, documented experiments*, not a customer. This repository is a lab bench rendered in code: experiments are the benchtop apparatus, the raw-data folders are the reagent shelves (some kept in the locked cabinet, never shipped), the provenance files are the logbook, and the publication guard is the protocol at the door deciding what may leave. If you go looking for an app's architecture — a server, a UI, a main loop — you will be confused. Look instead for the shape of a well-run lab, and it snaps into focus.

1.2 The top-level map

The repository is large — well over a thousand tracked files, some fifty experiments, as many papers — but its top level is a small, legible set of rooms:

Directory	What lives there
`experiments/`	~50 self-contained research units — the harness code, the GPU-sweep entry points, committed result summaries, and a provenance card each
`papers/`	~55 paper directories, each a `paper.md` plus figures, rendered to `paper.pdf`; a `pdf/` folder of shareable copies
`scripts/`	~85 operational scripts: the quality gate, the provenance generator, and dozens of one-off paper- and figure-builders
`tests/`	~55 test files, one per experiment or module
`formal/ontology-hs/`	a Haskell program — a typed "ontology gate" for one arc of experiments (Chapter 8)
`docs/`	the two canonical design docs, the verification index, and a dense archive of plans, reviews, and handoffs
`data/`, `artifacts/`	raw data and raw outputs — *gitignored*, never shipped (with one small committed exception)
`references/`	the full-text citation archive — also gitignored

1.3 The surprising absence: no installable package, no lockfile

An experienced engineer opening the project's configuration finds something startling. The `pyproject.toml` — the file that usually declares a Python project's dependencies — declares *almost nothing*: a name, a Python version, and linter settings. No dependency list. No lockfile pinning exact versions. There is **no monolithic installable package** at all. This looks, at first, like a serious omission.

It is a deliberate choice, and understanding it is the first real lesson in the codebase's philosophy. Dependencies are not installed once and shared; they are pulled *ephemerally, per invocation*. A command that needs PyTorch and NumPy spins up a throwaway environment with exactly those, runs, and disappears (via a tool called `uvx`, Chapter 4). The GPU sweeps bake their dependencies into per-experiment cloud images instead. The reasoning: the author's local machine runs an old Python, but the experiments need a newer one, so every entry point forces its own fresh, correct environment rather than relying on a shared install that might be wrong. It trades the convenience of "install once" for the guarantee that each experiment declares and gets exactly what it needs.

Smell — the version-pinning is scattered, and mostly loose

A mature engineer should flag the cost of this choice, which the codebase itself acknowledges. Because there is no central lockfile, the exact versions of dependencies live scattered across many call sites — the quality-check script, the reproduce script, and each GPU image, sometimes as loose ranges rather than exact pins. Two consequences: the same experiment could resolve to slightly different library versions in different places (a reproducibility risk), and there is no single place to audit or update the dependency set. Only the most recent, most careful experiments pin exact versions; the older ones drift. Chapter 9's refactor plan puts "pin dependencies once" near the top for exactly this reason.

1.4 Why the framing matters for you as a reader

Hold the instrument framing as you read on, because it is the key that decodes the rest. When you meet a script that regenerates provenance from committed files, ask "what honesty problem does this solve?" — not "what feature does this ship?" When you meet the split between gitignored raw data and committed summaries, read it as the lab's rule about what leaves the room. When you meet the elaborate GPU-launch safety checks of Chapter 5, read them as a lab's protocol against an expensive mistake. The repository is an argument, made in code, that AI-generated science can be trustworthy — and every file is a sentence in that argument.

1.5 What to carry forward

The repository is a scientific instrument, not an app — its product is trustworthy, reproducible records; the top level is a legible set of "rooms" (experiments, papers, scripts, tests, formal, docs, and gitignored raw-data cabinets); there is deliberately no installable package or central lockfile — dependencies are pulled ephemerally per run, which guarantees correctness but scatters version pins (a real smell); and the way to read every subsequent choice is to ask which honesty or reproducibility problem it solves.

CHAPTER 2

The Experiment Package and the Pipeline

2.1 One convention, repeated fifty times

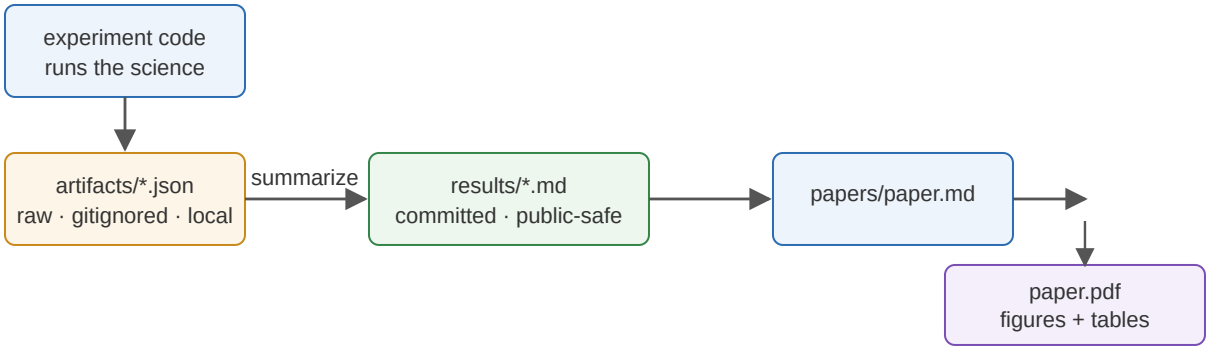
The codebase's coherence comes from a single repeated convention: the **experiment package**. Nearly every research unit under `experiments/` follows the same internal shape, so once you can read one, you can read all fifty. A typical package contains a hypothesis note, the harness code (the Python that actually runs the experiment), one or more GPU-sweep entry points, committed result summaries in a `results/` folder, an auto-generated provenance card, and sometimes a benchmark card.

Definition — the experiment package contract

An experiment package is expected to carry: a *hypothesis manifest* (what is being tested), *deterministic seeds* (so runs reproduce), a *positive target plus negative controls plus stress tests* (Chapter 3 of the science primer), *both accepted and rejected artifacts* (failures preserved, not deleted), a *discovery-regime audit* after each meaningful run, and the rule that *raw outputs stay in the gitignored artifact folder until they are summarized* into a committed, public-safe report. The package is a lab protocol expressed as a directory layout.

2.2 The pipeline: the spine of the whole repository

Data flows through every experiment along the same path, and this pipeline is the single most important thing to internalize about the codebase's operation:



The experiment-to-paper pipeline. Experiment code produces raw JSON dumps into the gitignored `artifacts/` cabinet; a summarizer reduces them to committed, public-safe Markdown in `results/`; the paper draws its numbers from those summaries; and a build script renders the paper (with figures and tables) to PDF. The load-bearing rule: *raw data never ships; only its vetted summary does.*

Intuition pump — the darkroom and the gallery

Think of an old photographer's workflow. The negatives (raw, fragile, private) live in the darkroom; they never go on the wall. From them the photographer produces prints — curated, finished, safe to display — and those go in the

gallery. You could not run the gallery without the darkroom, but the two are strictly separated, and only prints cross the threshold. The repository's `artifacts/` folder is the darkroom (raw JSON, gitignored, local-only); `results/` and the papers are the gallery (summarized, committed, public). The summarizer is the printing step. This separation is not tidiness — it is the mechanism that lets the whole thing be a *public* repository without ever leaking a raw negative.

2.3 A subtlety: "paper-primary" experiments

Not every experiment produces a committed `results/` summary. A large class of the newer "concern" experiments are *paper-primary*: one GPU sweep feeds directly into one paper, with *no* committed intermediate summary — the paper itself is the public record, and the raw JSON stays local. The provenance system encodes this distinction directly, tagging each experiment's status as "paper," "results," or "scaffold" depending on what public artifact it has reached. This is worth knowing because it means "where do I find this experiment's numbers?" has two different answers depending on the experiment's maturity — sometimes a results file, sometimes only the paper.

2.4 What the convention buys, and what it costs

The upside of one repeated package shape is enormous: any contributor (human or AI) can navigate an unfamiliar experiment instantly, because it looks like every other one; tooling can operate on all experiments uniformly (the provenance generator scrapes every package the same way); and the pipeline's darkroom/gallery split is enforced everywhere at once. The cost, which Chapter 9 develops, is *duplication*: because each package re-implements the same scaffolding by hand — the same seeding logic, the same "dict of pass/fail gates," the same summarization — the convention is maintained by copying rather than by a shared library. It is a convention held together by discipline, not by abstraction, and that is both its robustness and its debt.

2.5 What to carry forward

The codebase's coherence comes from one repeated "experiment package" shape (hypothesis, seeds, positive/controls/stress, accepted-and-rejected artifacts, provenance); the universal pipeline runs experiment code → gitignored raw JSON → committed summary → paper → PDF, with the load-bearing rule that raw data never ships; "paper-primary" experiments skip the summary and let the paper be the record; and the convention buys uniform navigability and tooling at the cost of hand-copied duplication rather than shared abstraction.

CHAPTER 3

Public-Safe by Construction

3.1 The problem a public research repo must solve

This repository is *public* — anyone can read it. But the science it runs touches things that must not be public: raw model outputs, full-text copies of copyrighted papers, API keys, large data dumps. A single careless commit could leak a secret or a gigabyte of raw logs. The naive solution — "be careful" — fails, because in a repository where an AI agent commits at high volume, "be careful" is not a mechanism. The codebase instead makes safety **structural**: it arranges things so that leaking is *hard by construction*, not merely against the rules.

Definition — defense in depth

Defense in depth is the security principle of stacking independent barriers, so that a failure of any one is caught by the next. The repository stacks three for public safety: (1) the `.gitignore` that prevents raw-data folders and secrets from being tracked at all; (2) a *publication guard* that re-checks every tracked file and fails the build if something unsafe slipped through; and (3) *export scripts* that whitelist exactly which fields of a raw sweep may become public. Three walls, each independent, guarding one boundary.

3.2 The three walls

Wall one — the ignore list. The `.gitignore` blocks the raw-data directories, the full-text citation archive, and every kind of credential file from ever entering version control. This is the first and cheapest barrier: files it covers are invisible to git.

Wall two — the publication guard. A small, dependency-free script scans every *tracked* file and fails if it finds a forbidden path, a file over ten megabytes, or anything matching a credential signature (the shapes of API keys and tokens). This catches what the ignore list missed — a raw file committed by mistake, a secret pasted into a source file. It runs as part of the quality gate, so a violation blocks the build.

In the code — the guard's detector is itself unit-tested

A detail that signals real engineering seriousness: the secret-detecting function is exposed as a testable unit and locked down by its own test. The test asserts the detector *flags* real vendor tokens and `api_key='...'` assignments, but does *not* flag innocent fixture credential *names* used in test code. In other words, the guard against missing a real secret is itself guarded against crying wolf on test fixtures. The safety net has a safety net. This is the texture of a codebase that treats its trust infrastructure as first-class, tested code rather than a hopeful afterthought.

Wall three — field-allowlist exporters. The most sophisticated barrier. When a raw GPU sweep (say, a 200-kilobyte JSON of every cell's full internal state) needs to become a public appendix, an export script does not copy it — it *rebuilds* a compact artifact containing only an explicit whitelist of safe fields. It even records the raw source's cryptographic hash and lists exactly which fields were *omitted*, so the reduction is itself auditable. A reader can reconstruct the paper's tables from the compact public artifact without ever seeing the raw logs, and can verify nothing was hidden that shouldn't be.

Intuition pump — the classified document and the redaction stamp

When a government releases a classified document, it does not hand over the original and trust the reader to look away from the secrets. It produces a *redacted* copy: specific passages blacked out, and often a note saying how many lines were removed and why. The field-allowlist exporter is a redaction machine for scientific data. It starts from the raw, sensitive negative and emits a copy containing only the pre-approved fields, stamped with a note of what was left out and a hash proving it came from the real original. The difference from ordinary "just don't commit the secret": redaction is a *positive whitelist* — only named-safe fields survive — so a newly-added sensitive field is excluded by default rather than leaked by oversight.

3.3 Why "by construction" beats "by policy"

The deep move here, and the reason it belongs in a book about honest AI-generated science, is the shift from *policy* to *construction*. A policy ("don't commit secrets") is a rule that depends on every contributor following it every time — and with a high-volume AI committer, "every time" is a lot of chances to fail. A construction (an ignore list plus an automated guard plus a whitelist exporter) makes the safe path the *default* and the unsafe path *blocked*. You do not have to remember to be safe; you have to actively defeat three independent barriers to be unsafe. For a repository whose commits come from a machine, this is the only kind of safety that actually holds.

3.4 What to carry forward

A public research repo must never leak raw outputs, full-text sources, secrets, or huge dumps; the codebase makes safety structural via defense in depth — the gitignore (blocks tracking), the publication guard (re-scans tracked files for forbidden paths/sizes/credential patterns, with its detector itself unit-tested), and field-allowlist exporters (redact raw sweeps to whitelisted fields, recording hashes and omissions); and the principle is safety by construction, not by policy — the only kind that holds when a machine is committing at volume.

PART II

Running the Science

An instrument is defined by how it runs. This part covers the two execution regimes — deterministic local runs and large rented-GPU sweeps — the striking spectrum of engineering maturity across the cloud sweeps, and the toolchain that turns a Markdown draft into a finished PDF without a heavyweight publishing system.

CHAPTER 4

Two Execution Regimes

4.1 Local, deterministic, and seeded

Small experiments and all "pure" analysis run **locally, on the CPU, deterministically**. "Deterministic" is the key word: given the same inputs and the same random seed, the code produces *byte-for-byte* the same result, every time, on any machine. This is not automatic — randomness in software is a common source of irreproducibility — and the codebase achieves it through unusual discipline.

In the code — no global randomness, ever

The determinism discipline is remarkably consistent: **there is no global random-number generator anywhere**. Every function that needs randomness is *handed* its own generator, seeded explicitly, rather than reaching for a shared global one. Several modules go further with a second layer: a top-level generator draws a per-trial seed, and each trial is then built from a fresh generator seeded with *that* — so a trial's content is independent of how many other things ran before it. Some experiments even run the whole thing *twice* and assert the two runs produced identical results, gating on "deterministic replay." The payoff: an experiment tagged "local" can be re-run by anyone with one command and will reproduce exactly, which is the bedrock of the reproducibility pillar (Chapter 7).

Intuition pump — the recipe that always yields the same cake

An ordinary recipe says "a pinch of salt, bake until done" — and two cooks get two different cakes. A *deterministic* recipe specifies every gram and every second, so any cook, any kitchen, gets the identical cake. Software randomness is the "pinch of salt": if the program grabs whatever random state happens to be lying around (a global generator), results wobble run to run. The codebase's rule — hand every function its own precisely-seeded generator — is the specified-to-the-gram recipe. It is more work to write, and it is why "reproduce this experiment" is a real promise here rather than a hope.

4.2 Ephemeral dependencies with uvx

Recall from Chapter 1 that there is no shared installed environment. Local runs get their dependencies through `uvx`, a tool that spins up a *throwaway* Python environment with exactly the requested libraries, runs the command inside it, and discards it. A quality-check run, for instance, invokes the tests inside a fresh environment carrying PyTorch, NumPy, scikit-learn, and the test framework — pinned to a specific Python version the code requires. The benefit is that each entry point declares and receives precisely its own needs, immune to whatever happens to be installed on the machine. The cost, flagged in Chapter 1, is that those declarations are scattered rather than centralized.

4.3 The cloud regime: Modal GPU sweeps

Anything large — training hundreds of neural networks, sweeping thousands of trials — cannot run on a laptop. These go to **Modal**, a service that runs your code on rented cloud machines, including GPUs, on demand. The pattern is uniform: a command dispatches a "sweep" — many independent runs across seeds and conditions — to Modal, which fans them out across many cloud workers, runs them in parallel, and collects the results. A typical dispatch

wraps the Modal command in **Doppler** (a secrets manager that injects the necessary cloud credentials into the local environment just for that command) and `uvx` (for the ephemeral dependencies), then names the experiment's sweep file and its output location.

Definition — a sweep, and sharding

A **sweep** is a batch of many experiment runs across a grid of settings — for example, 256 neural networks across 8 conditions and many seeds. Because the runs are independent, they can be **sharded**: split into groups, each group ("shard") sent to a separate cloud worker to run in parallel, with the results merged at the end. Sharding is what turns a computation that would take days on one machine into one that finishes in minutes across many — and how it is orchestrated safely is the subject of the next chapter.

4.4 The firewall between the regimes

The two regimes are kept strictly separate, and the boundary is a stated rule: local is for lightweight work (searching the code, running tests, linting, making figures); the cloud is required for anything heavy (multi-seed sweeps, neural training, architecture search). This protects the human director's laptop from being asked to do a supercomputer's job, and it keeps the expensive, credentialed, non-deterministic work in one clearly-marked place. The reproduce tooling (Chapter 7) respects this firewall: for local experiments it re-runs them outright; for cloud sweeps it prints the exact command to dispatch from an authorized machine, rather than pretending a laptop could reproduce a GPU sweep.

4.5 What to carry forward

Two execution regimes: local CPU runs are deterministic and seeded (no global RNG, per-trial seed isolation, double-run replay gates) so they reproduce byte-for-byte, with dependencies pulled ephemerally via `uvx`; and large work goes to Modal cloud GPUs as sharded sweeps wrapped in Doppler (secrets) and `uvx`. A firewall keeps light work local and heavy work in the cloud, and the reproduce tooling respects it.

CHAPTER 5

The Modal Maturity Gradient

5.1 A codebase caught in the act of growing up

One of the most instructive things about this repository — for anyone learning research infrastructure — is that its cloud-sweep code exists at three visibly different levels of maturity, all at once. You can watch the engineering learn, from a simple fan-out to a battle-hardened orchestrator, across the experiments. This chapter walks the three tiers, because each solves a real problem the previous one exposed, and the progression is a compact course in building reliable distributed compute.

5.2 Tier one — the simple fan-out

The earliest and simplest pattern: define one function that runs a shard of work, build a list of shard arguments differing only by a shard number, and map the function over the list. Modal runs them in parallel; the driver merges the results. Reproducibility comes from pure seed arithmetic (each shard's seed is derived from a base seed plus its shard number). The worker is deliberately self-contained — it re-implements the sweep inline rather than importing the repository — so there is nothing to synchronize. This is clean, legible, and entirely adequate for a bounded CPU sweep. Its limitation: if a worker dies partway, or the local machine disconnects, the work is lost — there is no memory of what finished.

5.3 Tier two — sharded GPU work

The next level adds GPUs and larger, longer runs. Each shard now handles all the cells for one model size, runs on a rented GPU with a time limit, and shares a cache for downloaded model weights. Dependency versions are range-pinned; the model weights ride the worker's environment. This handles real neural sweeps, but still has no checkpointing: a six-hour GPU shard that fails at hour five starts over from zero. For an occasional sweep that is tolerable; for the program's most important, most expensive experiments, it is not.

Intuition pump — the difference between a to-do list and a save point

Tier one and two are like doing a long task with no way to save your progress: interrupt it and you begin again. Anyone who has played a video game knows the difference a *save point* makes — you can be knocked out and resume from the last checkpoint rather than the start. The leap to tier three is the addition of save points, plus the coordination needed when many players work the same dungeon at once: a way to record which rooms are cleared, to claim a room so two players don't redo it, and to keep going even if one player's console crashes. That coordination is exactly what an expensive, resumable, parallel GPU sweep needs, and it is what the mature tier provides.

5.4 Tier three — the leased, checkpointed, resumable orchestrator

The reference implementation — worth studying as the codebase's engineering high-water mark — concentrates all the hard-won reliability machinery in one place, so the rest of the experiments can stay simple. It does several things at once, each answering a specific failure of the earlier tiers:

Mechanism	The problem it solves
"Exactly one action" gating	Accidental GPU spend. The launcher requires exactly one of dry-run / inspect / execute, requires a positive cost cap to execute, and — for confirmatory runs — requires the caller to paste back a "manifest id" that must byte-match the frozen configuration. You cannot fat-finger a thousand dollars of compute.
Exact version freeze	Silent dependency drift. Dependencies are read from an exact lockfile; the Modal client version is pinned and the run *refuses to start* if it differs; model weights are pinned to specific git revisions; and a pre-flight step re-checks the installed versions on the worker before any GPU spins up.
Checkpointing to a cloud volume	Lost work on failure. Each finished cell is written atomically to a persistent cloud volume; a re-run reuses any cell whose result passes an integrity check, so a failed sweep resumes instead of restarting.
Distributed leases	Two workers redoing the same cell. Workers claim a cell via an atomic "put-if-absent" operation on a shared dictionary — a lease — with expired leases carefully reclaimable, so parallel workers never duplicate work.
A remote orchestrator	The local machine disconnecting mid-sweep. Rather than the laptop driving the workers, a cloud function spawns and collects them, so closing the laptop cannot cancel the run.

In the code — safety proportional to stakes

Notice the design wisdom here: the elaborate machinery lives only where the stakes justify it. The cheap CPU sweeps stay at tier one — no checkpointing, no leases, because a failed CPU shard costs pennies and seconds. The expensive, confirmatory GPU experiments get the full tier-three treatment, because a failed run there costs real money and a botched launch could corrupt a headline result. This is exactly the right instinct: *don't pay the complexity tax where the risk doesn't warrant it.* A less mature codebase would either over-engineer everything or under-engineer the important thing; this one matches the armor to the threat.

5.5 The scientific-integrity payload of the launch gates

One feature of the tier-three launcher deserves special emphasis because it is where engineering meets epistemics (the science primer's territory). The "exactly one action" gating enforces a rule that a *smoke test can never promote a verdict*: a small calibration run is structurally prevented from being mistaken for the real confirmatory sweep, and the confirmatory sweep refuses to run unless its frozen configuration matches exactly what was pre-registered. The launch code, in other words, is not just protecting money — it is protecting the pre-registration discipline (science primer, Chapter 2) from being accidentally undermined by a rushed or misconfigured run. The gate against overclaiming lives partly in the deploy script.

5.6 What to carry forward

The cloud-sweep code spans three maturity tiers: a simple parallel fan-out (fine for cheap CPU work, but loses progress on failure); sharded GPU runs (handles neural sweeps, no checkpointing); and a leased, checkpointed, resumable orchestrator that adds accidental-spend gating, exact version freezing, cloud-volume checkpoints, distributed leases, and a remote driver — with the crucial wisdom that this armor is applied only where the stakes justify it, and that its launch gates also protect the pre-registration discipline from being undermined.

CHAPTER 6

From Markdown to PDF: the Paper Toolchain

6.1 Publishing without a publishing system

Academic papers are usually typeset in LaTeX — a powerful but heavy system, awkward to run in automated pipelines and on machines without a full installation. This repository takes a different route: it builds its PDFs with a lightweight, LaTeX-free toolchain, so a paper can be rendered anywhere Python runs, including on a cloud worker with nothing pre-installed. Understanding this toolchain matters because it is where the science becomes the shareable artifact — the last mile of the pipeline from Chapter 2.

6.2 The building blocks

At the toolchain's center is a small in-house library that wraps two mature tools: **reportlab** (a Python library that assembles PDFs programmatically — paragraphs, tables, images, page layout) and **matplotlib** (the standard Python plotting library, which produces the figures). The library offers a paper object with methods like "title," "abstract," "section," "paragraph," "figure," and "table," plus a house style (fonts, colors, zebra-striped tables) so every paper looks consistent. A figure-maker script reads an experiment's committed result JSON and emits the plots; a build script composes the document — prose, figures, tables — and writes the PDF, often to two locations at once (the paper's own folder and a shared shareable folder).

Intuition pump — a page layout program versus a print shop

LaTeX is like sending your manuscript to a full print shop: enormously capable, but you need the whole shop installed and running. The repository's approach is more like a focused page-layout program you carry on a USB stick: it does less, but it runs anywhere, needs no external installation, and fits inside an automated pipeline. For a codebase whose papers must sometimes be rendered on a bare cloud machine — and whose author wants "reproduce this paper" to be one command — the portable page-layout tool is the right trade. You give up LaTeX's typographic sophistication; you gain the ability to rebuild any paper anywhere, deterministically.

6.3 The most important distinction: where the numbers come from

Here is the detail that most repays attention, because it reveals the codebase's engineering maturity gradient in a single dimension — *where a paper's numbers originate*. There are two patterns, and the contrast between them is the clearest "old versus new" fault line in the repository:

Pattern	How numbers reach the PDF	Consequence
Mature (JSON-driven)	The build script reads the experiment's committed result JSON and renders figures and tables *from that data*.	A clean clone reproduces the paper's numbers from the committed evidence. The paper cannot silently disagree with the results.
Legacy (hardcoded)	The numbers are typed as literal values *inside the build script itself* — copied from the paper	The build script *is* the source of truth; nothing links it to the underlying results, so the two can drift apart

	text by hand.	undetected.

Smell — the flagship paper's numbers are welded into its build script

A mature engineer should note, with some alarm, that the *flagship* weakness paper's build script is of the legacy kind: it states outright that its numbers are "taken verbatim from the paper text," and inlines every value as a Python literal. Nothing is loaded from a result file; the script is the data. This means the single most important paper in the repository has no machine-checkable link between its published numbers and any experimental artifact — the exact gap the observability apparatus (Chapter 7) exists to close, left open in the most consequential place. The newer commitment-surface papers do it right, reading from committed JSON. The repository is visibly mid-migration from the legacy pattern to the mature one, and this is the highest-value migration still incomplete.

6.4 Why this is more than a style nit

It would be easy to dismiss "hardcoded numbers in a build script" as a minor untidiness. It is not, and seeing why sharpens the whole book's theme. The entire trust argument of this repository (Chapter 7, and the science primer) rests on *traceability*: every published claim should trace back through committed artifacts to the run that produced it. A paper whose numbers are hand-typed into its renderer breaks that chain at the last step — the PDF a reader sees is no longer provably derived from the evidence; it is derived from a human (or AI) transcription that could contain an error or a drift no tool would catch. For a program built to make AI-generated science verifiable, a hardcoded flagship is a crack in the foundation, and Chapter 9's top refactor recommendation — a declarative paper manifest that forces every number to originate in committed data — is aimed squarely at it.

6.5 What to carry forward

Papers are built LaTeX-free with a small in-house library over reportlab and matplotlib, so any paper renders anywhere Python runs; the crucial distinction is where numbers originate — the mature pattern reads them from committed result JSON (traceable), the legacy pattern hardcodes them in the build script (drift-prone); the flagship weakness paper is legacy, breaking the traceability chain at its most consequential point; and closing that gap — every number from committed data — is the repository's highest-value incomplete migration.

PART III

Keeping Code Honest

The instrument's credibility rests on its self-checking machinery: the quality gate, the provenance system, and the type-level guards that make a scientific mistake a compile error. This part covers how the codebase polices itself — and where that policing has gaps.

CHAPTER 7

The Quality and Provenance System

7.1 One command, five checks

Before committing substantive changes, a contributor runs a single quality-check wrapper that executes five stages in order: the full test suite (in a fresh, correctly-versioned environment); a byte-compile pass over all the code (which catches syntax errors even in files the type-checker skips); the publication guard (Chapter 3); a linter; and a type-checker. Any stage failing aborts the whole gate. This is a conventional continuous-integration battery — but with two unconventional twists worth noting.

In the code — the type-checker is declawed exactly where risk is highest

Here is a genuine tension a careful reader should sit with. The linter and type-checker are deliberately relaxed on the "experiment notebook" scripts and — critically — on *every* Modal GPU-sweep file. The rationale is understandable: these are research scripts, written fast, full of the mess that exploratory code accumulates. But the consequence is uncomfortable: the largest, *most expensive, most operationally dangerous* surface in the codebase — the scripts that spend real money on GPUs — has the *weakest* automated checking. A typo in a sweep's dispatch logic is exactly the kind of error a type-checker catches, and exactly the kind that is turned off there. Chapter 9 recommends narrowing this exclusion to at least cover the launch-and-dispatch logic, even if the inner GPU code stays unchecked.

7.2 The missing wall: no continuous integration

Smell — the quality gate is opt-in, not enforced

The single most consequential engineering gap in the repository: **there is no automated continuous-integration workflow that runs the quality gate on every change.** The gate exists and is excellent; but running it depends on the contributor *choosing* to run it locally before committing. The only automated GitHub workflow deploys the public websites. This means a change that breaks a test, leaks a secret past the guard, or fails the linter can be merged with a green appearance, because nothing automatically checked it. For a repository where an AI agent commits at high volume — the very situation where you most want an unbypassable gate — the gate is bypassable by omission. This is the first thing an incoming engineer should fix, and it is cheap: the gate is already written; it simply is not wired to run on pull requests.

7.3 Provenance: the lab notebook that regenerates itself

The provenance system is the codebase's answer to the trust problem, rendered as tooling. A generator script scrapes every experiment package and writes a provenance card recording its attribution, the exact run command, the seed, the pre-registration link, extracted gate signals, and a one-command reproduce recipe. The crucial property: the cards are *generated, not hand-written*, and the generator is "itself the reproducible artifact" — re-run it and every card is rebuilt from the committed files, so provenance cannot silently drift out of sync with reality. The generator also emits a machine-readable manifest of all experiments (mirrored into the public website) and a human-readable index.

Intuition pump — the git of scientific claims

Version control (git) works because it does not *ask* you what the code's history is — it *computes* the history from the commits themselves, so the record cannot lie about what happened. The provenance generator applies the same idea to scientific claims: it does not ask the experimenter to describe how a result was made; it computes the description from the committed files. A hand-written lab notebook is a claim about the work; a generated provenance card is a *derivation* from the work. In a program where the "scientist" is an AI, this shift — from asking to deriving — is what lets a reader trust the record without trusting the machine that wrote it.

7.4 One-command reproduce

The reproducibility pillar is made concrete by a dispatcher script: name an experiment and it either re-runs it locally (for deterministic CPU experiments, checking the output matches), rebuilds its paper PDF from committed numbers, or — for GPU/secret sweeps a laptop cannot run — prints the exact documented command to dispatch from an authorized machine. Its stated purpose is that "a wiped environment never forces a manual re-run." This is the tooling that makes the abstract promise "our science is reproducible" into a literal command a skeptic can type.

Smell — the audit layer can itself mislead

An honest accounting must note a subtle failure mode the program itself caught. The provenance generator extracts "gate signals" from result reports using pattern-matching over prose — and that scraper has been observed *misfiring*, misreading careful hedging language as a passing gate. This is the audit layer producing a false positive about its own subject. It is a small bug, but a philosophically loaded one: the machinery meant to make the science verifiable can, itself, be wrong about the science. The fix the program identifies — emit structured, machine-readable gate verdicts and score from *those*, rather than scraping human prose — is the right one, and it generalizes: trust infrastructure should read structured data, not natural language, wherever possible.

7.5 The honest ceiling of the whole system

Assemble the quality gate, the guard, the provenance generator, and the reproduce dispatcher, and you have a genuinely impressive trust apparatus — better than most research codebases possess. But its ceiling must be stated plainly, because it is the same ceiling the science and history primers reach: this machinery proves *the recipe* (what command was recorded, from what committed files), not *the meal* (that the numbers reflect an unbiased execution). The committed summaries are AI-written condensations of raw outputs that are themselves gitignored, so a reader usually cannot recompute a result from the raw rows. The tooling makes the process observable; it does not, by itself, make the results independently checkable. Closing that final gap — committing raw rows for flagship experiments, scoring from structured records, and inviting outside replication — is engineering the program has scoped but not finished.

7.6 What to carry forward

A one-command quality gate runs tests, compile-check, publication guard, linter, and type-checker — but the type-checker is relaxed exactly on the risky GPU-spend scripts, and, worst, the gate is opt-in with no CI enforcing it; the provenance system generates (not writes) self-reproducing cards, a machine manifest, and one-command reproduce, applying "derive, don't ask" to scientific claims; and the honest ceiling is that this proves the recipe, not the meal — the audit layer can even misread prose — so committing raw rows and inviting external replication is the unfinished work.

CHAPTER 8

Types as Contracts, and the Polyglot Boundary

8.1 Making a scientific mistake a compile error

The most sophisticated engineering idea in the codebase is one it uses sparingly but brilliantly: encoding a *scientific integrity requirement* as a *type*, so that violating it becomes a programming error the tools catch before the code ever runs. This deserves careful explanation, because it is where software engineering and the science primer's methodology fuse into one thing.

Definition — types as contracts

A **type** in a program is a promise about what kind of value something is — a number, a string, a specific structured object. A type-checker verifies these promises without running the code. The powerful move: define *distinct types for things that must never be confused*, so that mixing them is not a subtle runtime bug but an immediate, un-runnable type error. The type system becomes a *contract* the compiler enforces.

In the code — a leakage a scientist fears, made a type error

The reference example is a pure, dependency-free core module underlying one of the flagship experiments. Its stated purpose: to have "no Modal, torch, transformers, or PEFT dependency, so a regularizer cannot accidentally acquire held-out truth labels." Read that again — the module is designed so that a specific *scientific* mistake (a training component secretly gaining access to the answer it is supposed to predict, invalidating the result) is *structurally impossible*, because the code that would need to touch the forbidden data simply cannot import it. It goes further, defining *distinct types* for a "supervised exposure" and a "consistency exposure" — two things that must never be mixed — so that confusing them is a **type error, not a runtime bug**. The data-leakage a scientist lies awake worrying about is caught by the type-checker at author time. And the experiment's tests re-declare the frozen constants independently and cross-check them, so the test suite is itself a witness that the integrity conditions hold.

Intuition pump — the blood-bank labels that can't be mixed up

A blood bank does not rely on technicians being careful not to mix incompatible blood types; it engineers the incompatibility into the process — different labels, different fridges, checks that physically refuse a mismatched transfusion. The catastrophe is designed out, not policed. The codebase's distinct types for "supervised" and "consistency" data are blood-bank labels: the compiler physically refuses to let the two touch, so the catastrophe (a leak that silently invalidates a headline result) cannot occur by oversight. This is the highest form of the "safe by construction" principle from Chapter 3, applied not to secrets but to scientific validity itself. Where the codebase does this, it is genuinely excellent engineering.

8.2 The polyglot boundary: Python meets Haskell

One arc of experiments needed to check whether a candidate agent "body" (a set of architectural components) is *admissible* under a web of dependency and resource rules. The codebase makes an interesting language choice here: it writes this checker not in Python but in **Haskell**, a language whose exhaustive pattern-matching and algebraic data

types make such rule-checking natural and hard to get wrong. The Haskell program models a body as a set of components, computes rule violations from a dependency table, and emits its verdict as JSON — one line per checked body.

In the code — a clean, degradable foreign-language boundary

How the two languages talk is a small model of good engineering. The Python side locates the Haskell program, runs it as a subprocess, and parses its line-delimited JSON output into strongly-typed, validated objects — checking, for instance, that a "resource cost" is genuinely an integer and not accidentally a boolean. Crucially, it distinguishes two failure modes with separate exceptions: "the checker could not run at all" (Haskell isn't installed) versus "the checker ran but returned garbage." And when the Haskell toolchain is simply absent, the Python side *soft-degrades* — it returns a "cannot verify" result rather than crashing, and refuses to silently convert a missing checker into a passing verdict. The interchange contract — line-delimited JSON, strictly validated on the Python side, with the foreign toolchain treated as optionally-absent — is a clean, testable model for crossing a language boundary, and the test suite can stub the subprocess to exercise it without Haskell present.

8.3 Why the polyglot choice is defensible

Reaching for a second language is a decision that should always be questioned — each language added is a tax on every future contributor. Here it is defensible for a specific reason: the admissibility rules are exactly the kind of thing (exhaustive case analysis over a fixed set of components, with the compiler guaranteeing no case is missed) that Haskell's type system makes *safer* than the equivalent Python would be. The rules are a formal specification, and Haskell lets that specification be machine-checked for completeness. The clean JSON boundary and the soft-degradation keep the tax bounded: the rest of the codebase does not need to know Haskell exists, and works fine when it is absent. It is a considered trade, not a whim — though a mature reviewer would still ask whether the same guarantees could be had in Python with less operational complexity, and the honest answer is "mostly, but less cleanly."

8.4 What to carry forward

The codebase's most sophisticated idea is encoding scientific-integrity requirements as types — a pure core that *cannot import* forbidden data, and distinct types for exposures that must never mix, so a result-invalidating leak becomes a compile error (safe-by-construction applied to validity itself); and it crosses into Haskell for one arc's admissibility checker, connected by a clean, strictly-validated, soft-degrading JSON boundary — a defensible polyglot choice where the type-level completeness guarantee earns its operational tax.

PART IV

A Mature Verdict

An honest engineering review ends with a balance sheet: what to imitate, what to fix, and a concrete plan. This final chapter delivers it — the genuine strengths worth copying into your own projects, the smells and debt a new engineer should expect, and a prioritized refactor path toward a codebase that scales without multiplying its one-off scripts.

CHAPTER 9

Strengths, Smells, and the Refactor Path

9.1 What to imitate

Set aside the research and judge purely as engineering: several patterns here are worth stealing for any project that values correctness.

- **Reproducibility as a first-class feature.** No global randomness, per-trial seed isolation, double-run replay gates, and a self-regenerating provenance system. "Reproduce this" is a real command, not a hope.
- **Safety by construction.** The three-wall public-safety stack (ignore list, tested guard, whitelist exporters) makes leaking hard rather than merely forbidden — the right model whenever a high-volume or automated committer is involved.
- **Types that enforce the domain's real invariants.** Making a data-leak a compile error is the single best idea in the codebase and generalizes far beyond research: encode the mistake you most fear as a type.
- **Safety proportional to stakes.** The Modal maturity gradient — simple fan-out for cheap work, a hardened orchestrator only for expensive confirmatory runs — is disciplined engineering judgment, neither over- nor under-building.
- **Test-the-test.** Adversarial fixtures that must make gates *fail*, and a publication-guard detector with its own false-positive test. The safety nets have safety nets.

9.2 The smells, ranked by how much they matter

Smell	Why it matters
1. No CI enforcing the quality gate	The excellent gate is opt-in; a broken or unsafe change can merge green. The highest-value, lowest-effort fix — the gate already exists, it just isn't wired to run on pull requests.
2. Hardcoded numbers in build scripts (esp. the flagship)	Breaks the traceability chain from published number to committed evidence at the last step — the exact thing the trust apparatus exists to prevent, left open in the most important paper.
3. The 50+ one-off paper/figure scripts	Each new paper spawns a new ~300-line bespoke script with no shared schema; numbers can drift, and the surface grows linearly with the science.
4. No dependency lockfile	Version pins scattered across call sites, mostly loose; the same experiment can resolve to different library versions in different places.
5. Weakest static checking on the riskiest (GPU-spend) scripts	The money-spending dispatch logic is exactly where a type error is costly, and exactly where the type-checker is turned off.
6. Hand-copied duplication across experiments	The gate idiom, the seeding helpers, the candidate-construction logic are re-implemented per experiment rather than shared — robustness by discipline, not abstraction.

7. Machine-specific config in many files	Hardcoded local paths and deploy IDs scattered through docstrings and workflows; portability requires editing many files.

Intuition pump — the difference between a workshop and a factory

A brilliant craftsperson's workshop produces superb one-off pieces, each made bespoke, tools and jigs improvised per job. It works because the craftsperson holds the standards in their head. A factory produces the same quality *repeatably* by encoding the standards into shared jigs, fixtures, and a line — so the hundredth piece is as good as the first without heroics. This codebase is a superb workshop: its individual experiments are excellent, but its quality lives in discipline (and in the AI agents' consistency) rather than in shared abstractions. The refactor path is the workshop-to-factory transition: turn the repeated hand-work — the gate idiom, the paper-building, the seeding, the dependency pins — into shared fixtures, so that scaling to the next fifty experiments does not mean writing the same scaffolding fifty more times.

9.3 The refactor path, in priority order

A concrete plan

1. **Wire the quality gate into CI.** Run the existing gate on every pull request. Start cheap (guard + compile-check + a fast test subset). This is the biggest trust improvement for the least effort — it makes the whole quality system unbypassable.
2. **A declarative paper manifest.** Replace the 50+ bespoke build/figure scripts with one generic renderer that consumes a small per-paper config (source JSON paths, figure specs, table specs, prose). This kills smells 2 and 3 at once: every number is *forced* to originate in committed data, and the script count collapses toward one engine plus N configs.
3. **A shared "Gate" abstraction.** One small type for a named pass/fail gate carrying its evidence, plus a standard serialization. Migrate the hand-rolled per-experiment gate dicts onto it, so gates become uniformly machine-readable — which also fixes the provenance scraper's prose-misreading (Chapter 7) by giving it structured records to read.
4. **A shared experiment scaffold.** Extract the duplicated seeding, summarization, and candidate-construction helpers into a base module every experiment builds on. Robustness by abstraction, not by copying.
5. **Pin dependencies once.** A single lockfile that the quality gate, the reproduce script, and the cloud images all read, replacing the scattered loose pins. Adopt the exact-pin discipline the newest experiments already use, repository-wide.
6. **Type-check the launch logic.** Narrow the static-analysis exclusion so that at least the argument-parsing and dispatch-gating of the GPU scripts are checked, even if the inner numeric code stays excluded. The money-spending logic is where a type error hurts most.
7. **Externalize machine-specific config.** Move local paths and deploy IDs into one config file, so run commands are host-agnostic.

9.4 The honest bottom line

As engineering, this is a strong, unusual codebase whose best ideas — reproducibility as a feature, safety by construction, types enforcing scientific validity, armor proportional to stakes — are genuinely worth imitating, and are more disciplined than a great deal of production software. Its weaknesses are the characteristic weaknesses of a fast-moving research workshop authored by AI agents: an opt-in rather than enforced quality gate, a flagship paper whose numbers aren't traceable, and a proliferation of one-off scripts held together by discipline rather than abstraction. None of these is deep; all are the kind of debt that accumulates when a small team (here, a human plus tireless agents) moves fast and prioritizes results over infrastructure. The refactor path is clear, incremental, and mostly cheap — and the first step, wiring the existing quality gate into CI, would do more for the codebase's trustworthiness than any new feature. The instrument is well-built; it now needs the factory fixtures that let it stay well-built across the next hundred experiments.

9.5 What to carry forward

Imitate: reproducibility-as-feature, safety-by-construction, types-enforce-validity, armor-proportional-to-stakes, test-the-test. Fix, in order: wire the quality gate into CI (biggest win, least effort); a declarative paper manifest to force numbers from committed data and collapse the one-off scripts; a shared Gate abstraction and experiment scaffold to replace hand-copied duplication; a single dependency lockfile; type-checking on the GPU launch logic; and externalized config. The instrument is excellent; it needs the workshop-to-factory transition to scale.